

The Realm of Primitive Recursion

Harold Simmons

Department of Mathematics, University of Aberdeen, Edward Wright Building, Dunbar Street,
Aberdeen AB9 2TY, UK

An integer function

$$f: \mathbb{N}^2 \rightarrow \mathbb{N},$$

(where $\mathbb{N} = \{0, 1, 2, 3, \dots\}$) is obtained from two functions

$$g: \mathbb{N} \rightarrow \mathbb{N}, \quad h: \mathbb{N}^3 \rightarrow \mathbb{N}$$

by an immediate application of primitive recursion if for all $r, x \in \mathbb{N}$ we have

$$\begin{cases} f(0, x) = g(x) \\ f(r+1, x) = h(r, x, f(r, x)) \end{cases} \quad (1)$$

Here r is the *active variable*. This mode of construction has many different refinements. For instance there is primitive recursion with substitution for parameters

$$\begin{cases} f(0, x) = g(x) \\ f(r+1, x) = h(r, x, f(r, k(r, x))) \end{cases} \quad (2)$$

where g, h (as above) and $k: \mathbb{N}^2 \rightarrow \mathbb{N}$ are given functions. More generally we may be allowed to compute $f(r+1, x)$ by an explicitly given recipe from the previously defined functions $f(0, \cdot), f(1, \cdot), \dots, f(r, \cdot)$.

It is known that these and many more complex modes of construction are reducible to primitive recursion, i.e., the defined function f is primitive recursive in the given functions g, h, k etc. The question is, why is this so, and is there an illuminating proof of this fact? Furthermore, just how far can these refinements be pushed and still remain within the realm of primitive recursion?

The standard text books are not of much help on these questions. Rarely do they go beyond rather modest extensions of primitive recursion, and these are justified by somewhat ad hoc means. The most complete discussion of these matters is given by Rózsa Péter in her classic book [2]. Many complicated modes of construction are reduced to primitive recursion; however the proofs seem to be rather technical trickery, and do not readily give any insight as to why such

reductions are possible, nor do they suggest how other modes of construction could be reduced to primitive recursion.

Here I will present a proof which appears to cover all known reductions and does offer some explanation as to why the reduction works. The proof will at least make it clear how to work out the details of any particular case without too much trouble. Of course, other proofs of reduction to primitive recursion can be dug out of the literature. For instance, such a reduction can be extracted from [3]. However, the method I give here seems more natural, and is certainly less technically involved.

This proof is the result of reflecting on the various reductions given in [2] with the aim of finding a common strategy behind the methods. The crucial step to take is hinted at in [2] but never fully exploited. The trick is to analyse the *functional* form of the defining clause in the recursion step.

Let $\mathbb{F}$ be the set of all functions $\mathbb{N} \rightarrow \mathbb{N}$. For a given function $g \in \mathbb{F}$ and *functional*

$$H: \mathbb{N}^2 \times \mathbb{F} \rightarrow \mathbb{N}$$

consider the function f defined by

$$\begin{cases} f(0, x) = g(x) \\ f(r+1, x) = H(r, x; f(r, \cdot)) \end{cases} \quad (3)$$

for $r, x \in \mathbb{N}$. (Note how I have used the semicolon to separate the integer variables from the function variable in H .) All of the standard refinements of primitive recursion take the shape (3) for some explicitly given, and rather elementary, functional H . Thus our problem is to locate a sufficiently broad, but effective, class $\mathcal{H}$ of functionals for which:

(i) Each known refinement of primitive recursion has the form (3) for some $H \in \mathcal{H}$.

(ii) For each $H \in \mathcal{H}$ of the appropriate arity (i.e., $\mathbb{N}^2 \times \mathbb{F} \rightarrow \mathbb{N}$), the function f defined by (3) is primitive recursive in g .

Furthermore (for this is the whole point of the exercise) we require an illuminating proof of (ii).

This paper will provide such a class $\mathcal{H}$.

Before I begin the proof I should point out that what is done here is a reduction to course of values recursion. In the context of integer functions (which is the one considered here) course of values recursion and primitive recursion are equivalent, however in more general contexts they are not.

I originally worked out this proof solely for my own enlightenment, however a remark by Alan Bundy alerted me to the possible wider interest of the method, particularly in Theoretical Computer Science. The reaction of several other people, particularly Helmut Schwichtenberg and John Tucker, reinforced the view. I am grateful to them and several other people for their helpful comments. I am particularly grateful to the Centre for Theoretical Computer Science at the University of Leeds which organized the workshop at which I first presented this proof.

1. A Fundamental Obstruction

In our search for a class $\mathcal{H}$, the first thing to do is check on the class of primitive recursive functionals. These are the functionals built up from the successor and projection functions and the application functionals by means of composition and primitive recursion. We will require only a minimal acquaintance with these functionals, but the reader who requires a deeper understanding may consult [1].

The application functional

$$\text{App} : \mathbb{N} \times \mathbb{F} \rightarrow \mathbb{N}$$

given by

$$\text{App}(x; g) = g(x)$$

(for $x \in \mathbb{N}$ and $g \in \mathbb{F}$) is trivially primitive recursive (for it is an initial functional), hence so is the functional

$$\text{It} : \mathbb{N}^2 \times \mathbb{F} \rightarrow \mathbb{N}$$

given (for $r, x \in \mathbb{N}$ and $g \in \mathbb{F}$) by

$$\text{It}(0, x; g) = \text{App}(x; g)$$

$$\text{It}(r+1, x; g) = \text{App}(\text{It}(r, x; g); g)$$

(for it is obtained from App by an immediate application of primitive recursion).

Thus, setting

$$\Delta(x; g) = \text{It}(x, x; g)$$

(for $x \in \mathbb{N}$ and $g \in \mathbb{F}$) we obtain a primitive recursive functional

$$\Delta : \mathbb{N} \times \mathbb{F} \rightarrow \mathbb{N}.$$

Now, for arbitrary $g \in \mathbb{F}$, consider the function f defined by

$$f(0, x) = g(x)$$

$$f(r+1, x) = \Delta(x; f(r, \cdot)).$$

This is a simplified form of mode (3), so we should ask: Is f primitive recursive in g ?

We easily check that $\text{It}(r, \cdot; g)$ is the $r+1$ -fold iterate g^{r+1} of g , so if we define the diagonalization g^Δ of g by

$$g^\Delta(x) = \Delta(x; g) = g^{x+1}(x)$$

(for $x \in \mathbb{N}$) then we have

$$f(r, x) = g^\Delta \cdots \Delta(x)$$

the r -fold diagonalization of g . But, by the well-known Ackermann observation, the function $x \mapsto f(x, x)$ eventually dominates every function primitive recursive in g . Thus f cannot be primitive recursive in g .

This shows that the class of primitive recursive functionals cannot be a candidate for $\mathcal{H}$, for it is disqualified by requirement (ii). However, it certainly satisfies (i) so our search for $\mathcal{H}$ can be restricted to subclasses of primitive recursive functionals.

The problem is: What distinguishes the very simple construction of the function f above from the apparently much more complicated modes of construction which remain inside primitive recursion? The solution is the diagonalization used to construct Δ . This mixes up a previously active variable (r) with a dormant variable (x). However, if we are prepared to quarantine the active variables from interference by the other integer variables (so preventing the diagonalization trick) then everything will work smoothly. The precise restrictions needed are given in Sect. 4.

2. The Base Functional and Coding Paraphernalia

We start from a given fixed functional

$$\mathcal{E} : \mathbb{N}^p \times \mathbb{N}^q \times \mathbb{F}^r \rightarrow \mathbb{N}$$

of $p+q$ integer variables and r function variables. As indicated there are two kinds of integer variables; the first p of these are called *control variables* and will play a special role in the following development.

A typical value of $\mathcal{E}$ has the form

$$\mathcal{E}(l_1, \dots, l_p; u_1, \dots, u_q; f_1, \dots, f_r).$$

Here $l_1, \dots, l_p$ are the control variables; $u_1, \dots, u_q$ are the remaining integer variables; and $f_1, \dots, f_r$ are the function variables. We often abbreviate this value to

$$\mathcal{E}(\dots, l_p, \dots; \dots, u_q, \dots; \dots, f_r, \dots),$$

where

$$1 \leq i \leq p, \quad 1 \leq j \leq q, \quad 1 \leq k \leq r,$$

and even more often to $\mathcal{E}(\underline{l}; \underline{u}; \underline{f})$. Notice how again I have used semicolons to separate the three different kinds of variables.

Let $T = p + q + r$. There are several different ways in which T -sequences from $\mathbb{N}$ may be effectively coded by members of $\mathbb{N}$. Choose one of these and for each $y \in \mathbb{N}$ and $1 \leq t \leq T$ let

$$y^t = \text{the } t^{\text{th}} \text{ term of the sequence coded by } y.$$

It is convenient to assume that

$$0^t = 0 \quad \text{and} \quad y^t < y$$

for all $1 \leq t \leq T$ and $0 \neq y \in \mathbb{N}$. This will allow us a certain mode of recursive construction. Note that in particular $1^t = 0$. We also assume that all the functions $y \mapsto y^t$ are primitive recursive.

It is not important which sequence coding method is used, but for definiteness the reader may think of the one based on prime factorization.

Each T -sequence $y_1, y_2, \dots, y_T$ from $\mathbb{N}$ has a canonical code

$$\#(y_1, y_2, \dots, y_T)$$

so that

$$(\#(y_1, y_2, \dots, y_T))^t = y_t$$

for each $1 \leq t \leq T$. We assume that this code is never 0, and that the function $\#$ is primitive recursive.

3. The Ξ -Catalogue

We use the base functional Ξ and the coding machinery to produce a list $(\{y\} | y \in \mathbb{N})$ – called the Ξ -catalogue – of functions $\{y\} \in \mathbb{F}$ each of which is primitive recursive in Ξ . Writing $y(x)$ for the value of $\{y\}$ at $x \in \mathbb{N}$ (i.e., $y(x) = \{y\}(x)$), we define $\{y\}$ recursively by

$$0(x) = x$$

and

$$y(x) = \Xi(\dots, y^i, \dots; \dots, y^{p+j}(x), \dots; \dots, \{y^{p+q+k}\}, \dots)$$

for each $y \neq 0$ where

$$1 \leq i \leq p, \quad 1 \leq j \leq q, \quad 1 \leq k \leq r.$$

Thus $\{0\}$ is the identity function id . Note also the different roles played by the three different kinds of variables.

The binary function

$$(y, x) \mapsto y(x)$$

is primitive recursive in Ξ . This, of course, is verified by course of values recursion, not by straight primitive recursion.

We also use the sequence coding function $\#$ in a similar way to obtain a second binary function. Thus, for $y, z \in \mathbb{N}$ with $z \neq 0$, we set

$$0 * y = y$$

and

$$z * y = \#(\dots, z^i, \dots; \dots, z^{p+j} * y, \dots; \dots, z^{p+q+k}, \dots).$$

Again the binary function

$$(z, y) \mapsto z * y$$

is primitive recursive.

These two binary functions are intimately connected.

3.1. Lemma. For each $y, z \in \mathbb{N}$

$$\{z\} \circ \{y\} = \{z * y\},$$

(where “ $\circ$ ” indicates function composition).

Proof. We proceed by induction on z . Since $\{0\} = \text{id}$ we have

$$\{0\} \circ \{y\} = \{y\} = \{0 * y\}$$

so it remains to verify the induction step across z . To this end let $w = z * y$ and

$$L = \{z\} \circ \{y\}, \quad R = \{z * y\} = \{w\},$$

(where $z \neq 0$). For each $1 \leq t \leq T$ we have

$$w^t = \begin{cases} z^t & \text{if } p+q < t \leq T \\ z^t * y & \text{if } p < t \leq p+q \\ z^t & \text{if } 1 \leq t \leq p \end{cases}$$

and the induction hypothesis gives

$$z^t(y(x)) = (z^t * y)(x) = w^t(x)$$

for each $p < t \leq p+q$ and $x \in \mathbb{N}$. Thus

$$\begin{aligned} L(x) &= \mathcal{E}(\dots, z^i, \dots, \dots, z^{p+j}(y(x)), \dots, \dots, z^{p+q+k}, \dots) \\ &= \mathcal{E}(\dots, w^i, \dots, \dots, w^{p+j}(x), \dots, \dots, w^{p+q+k}, \dots) = R(x) \end{aligned}$$

as required. $\square$

The reader may like to check that the operation $*$ is associative.

4. The $\mathcal{E}$ -Functionals

We consider functionals of the type

$$A: \mathbb{N}^a \times \mathbb{N}^b \times \mathbb{F}^c \rightarrow \mathbb{N},$$

where a, b, c are arbitrary members of $\mathbb{N}$ (and at least one of which is non-zero). As a convenient way of indicating these arities we say A has *rank* (a, b, c) . In particular the base functional $\mathcal{E}$ has rank (p, q, r) . A typical value of A has the form

$$A(\underline{l}; \underline{u}; \underline{f}),$$

where $\underline{l} \in \mathbb{N}^a$, $\underline{u} \in \mathbb{N}^b$ and $\underline{f} \in \mathbb{F}^c$. As with $\mathcal{E}$ we refer to $\underline{l}$ as the control variables of A . These are treated with more respect than the other integers variables $\underline{u}$.

We are concerned with a class of such functionals A built up, in the manner of the primitive recursive functionals, from certain initial functionals (comprising $\mathcal{E}$ and some bookkeeping functionals) using a restricted form of composition and primitive recursion. The crucial restriction is that the control variables must not be interfered with by the other variables. Thus the diagonalization trick which enables us to leap out of certain well closed classes will not be possible.

The details are as follows:

First of all there are three kinds of initial $\mathcal{E}$ -functionals.

(0) The functional $\mathcal{E}$ is itself a $\mathcal{E}$ -functional.

(1) For each $a, b, c \in \mathbb{N}$ with $b \neq 0$, and each $1 \leq j \leq b$ there is a projection functional Proj of rank (a, b, c) where

$$\text{Proj}(\underline{l}; \underline{u}; \underline{f}) = u_j$$

for all appropriate $\underline{l}$, $\underline{u}$, and $\underline{f}$.

(2) For each $a, b, c \in \mathbb{N}$ with $b \neq 0 \neq c$, and each $1 \leq j \leq b$ and $1 \leq k \leq c$, there is an application functional App of rank (a, b, c) where

$$\text{App}(\underline{l}; \underline{u}; \underline{f}) = f_k(u_j)$$

for all appropriate $\underline{l}$, $\underline{u}$, and $\underline{f}$.

Secondly there are two allowable modes of construction. We deal with these one at a time.

Allowable composition: Let A be a given functional of rank (a, b, c) ; for each $1 \leq i \leq a$ let

$$\lambda_i: \mathbb{N}^l \rightarrow \mathbb{N}$$

be a given primitive recursive function; for each $1 \leq j \leq b$ let B_j be a given functional of rank (l, m, n) ; and for each $1 \leq k \leq c$ let C_k be a given functional of rank $(l, 1, n)$. We compose these to obtain a functional D of rank (l, m, n) . Thus for each

$$\underline{m} \in \mathbb{N}^l, \quad \underline{v} \in \mathbb{N}^m, \quad \underline{g} \in \mathbb{F}^n$$

we set

$$D(\underline{m}; \underline{v}; \underline{g}) = A(\underline{l}; \underline{u}; \underline{f}),$$

where for each $1 \leq i \leq a$, $1 \leq j \leq b$, $1 \leq k \leq c$

$$\begin{aligned} l_i &= \lambda_i(\underline{m}) \\ u_j &= B_j(\underline{m}; \underline{v}; \underline{g}) \\ f_k(\cdot) &= C_k(\underline{m}; \cdot; \underline{g}). \end{aligned}$$

The important restrictions here are that the control variables $\underline{l}$ depend only on the control variables $\underline{m}$, and the functions $\underline{f}$ are independent of the integer variables $\underline{v}$.

Allowable recursion: Let A and B be given functionals of ranks (a, b, c) and $(a, b+1, c)$ respectively, and let $1 \leq v \leq a$ be a fixed control position. We obtain a functional C of rank (a, b, c) from A and B primitively recursively where the active variable is the control variable l_v . Thus for

$$l_1, \dots, l, \dots, l_a \in \mathbb{N}, \quad \underline{u} \in \mathbb{N}^b, \quad \underline{f} \in \mathbb{F}^c$$

we have

$$\begin{aligned} C(l_1, \dots, 0, \dots, l_a; \underline{u}; \underline{f}) &= A(l_1, \dots, 0, \dots, l_a; \underline{u}; \underline{f}) \\ C(l_1, \dots, l+1, \dots, l_a; \underline{u}; \underline{f}) &= B(l_1, \dots, l, \dots, l_a; \underline{u}; \underline{v}; \underline{f}), \end{aligned}$$

where

$$v = C(l_1, \dots, l, \dots, l_a; \underline{u}; \underline{f}).$$

Here the distinguished control values are all in the v^{th} position.

Let us now formally introduce the $\mathcal{E}$ -functionals.

4.1. Definition. The $\mathcal{E}$ -functionals are those functionals built up from the initial $\mathcal{E}$ -functionals by allowable compositions and allowable recursions.

At this point the reader may like to turn to the final section (Sect. 7) where I give a couple of examples.

5. Indicial Functions

Each $\mathcal{E}$ -functional A of rank (a, b, c) gives us a whole family of functions in $\mathbb{F}$. Thus let

$$\underline{l} \in \mathbb{N}^a, \quad \underline{u} \in \mathbb{N}^b, \quad \underline{d} \in \mathbb{N}^c$$

be fixed sequences of parameters, and for each $x \in \mathbb{N}$ let

$$a(x) = A(\underline{l}; \underline{u}(x); \{\underline{d}\}).$$

Here we are making use of the $\mathcal{E}$ -catalogue in the second and third blocks of arguments in A . The crux of our argument is that such a function $a(\cdot) \in \mathbb{F}$ is itself always in the $\mathcal{E}$ -catalogue. Furthermore $a(\cdot)$ depends on the parameters $\underline{l}, \underline{u}, \underline{d}$ in a primitive recursive fashion.

Let us say a function

$$\alpha: \mathbb{N}^a \times \mathbb{N}^b \times \mathbb{N}^c \rightarrow \mathbb{N}$$

is *indicial* for A if

$$A(\underline{l}; \underline{u}(x); \{\underline{d}\}) = \{\alpha(\underline{l}; \underline{u}; \underline{d})\}(x)$$

for all appropriate $\underline{l}, \underline{u}, \underline{d}$, and x . For instance the coding function $\#$ is indicial for $\mathcal{E}$ since

$$\mathcal{E}(\underline{l}; \underline{u}(x); \{\underline{d}\}) = \{\#(\underline{l}; \underline{u}; \underline{d})\}(x).$$

Similarly for the projection functional Proj and the application function App [i.e., (1) and (2) of the previous section] we have

$$\text{Proj}(\underline{l}; \underline{u}(x); \{\underline{d}\}) = \{u_j\}(x)$$

$$\text{App}(\underline{l}; \underline{u}(x); \{\underline{d}\}) = \{d_k\}(u_f(x)) = \{d_k * u_j\}(x)$$

so both of these have indicial functions. This phenomenon is universal for $\mathcal{E}$ -functionals.

5.1. Proposition. *Each $\mathcal{E}$ -functional A has a primitive recursive indicial functional α . Furthermore α is effectively obtainable from a construction of A .*

Proof. We proceed by induction on the construction of A . The examples given above deal with the initial cases, so it remains to take the induction steps across allowable composition and recursion.

Suppose first that D [of rank (l, m, n)] is an allowable composite of A [of rank (a, b, c)] and appropriate λ_i , B_j , and C_k , as in Sect. 4. Let α , β_j , and γ_k be the assumed primitive recursive indicial functions of A , B_j , and C_k , respectively. For

$$\underline{m} \in \mathbb{N}^l, \quad \underline{v} \in \mathbb{N}^m, \quad \underline{e} \in \mathbb{N}^n$$

let

$$\delta(\underline{m}; \underline{v}; \underline{e}) = \alpha(\underline{\lambda}(\underline{m}); \underline{\beta}(\underline{m}; \underline{v}; \underline{e}); \underline{\gamma}(\underline{m}; 0; \underline{e}))$$

so clearly δ is a primitive recursive function

$$\delta: \mathbb{N}^l \times \mathbb{N}^m \times \mathbb{N}^n \rightarrow \mathbb{N}.$$

We show that δ is indicial for D .

Thus for $\underline{m}, \underline{v}, \underline{e}$ as above, and $x \in \mathbb{N}$ we have

$$D(\underline{m}; \underline{v}(x); \{\underline{e}\}) = A(\underline{l}; \underline{u}(x); \underline{f}),$$

where

$$\begin{aligned} \underline{l} &= \underline{\lambda}(\underline{m}) \\ \underline{u}(x) &= \underline{B}(\underline{m}; \underline{v}(x); \{\underline{e}\}) = \{\underline{\beta}\}(x) \\ \underline{f}(x) &= \underline{C}(\underline{m}; 0(x); \{\underline{e}\}) = \{\underline{\gamma}\}(x), \end{aligned}$$

where

$$\underline{\beta} = \underline{\beta}(\underline{m}; \underline{v}; \underline{e}), \quad \underline{\gamma} = \underline{\gamma}(\underline{m}; 0; \underline{e}).$$

Thus

$$\begin{aligned} D(\underline{m}; \underline{v}(x); \{\underline{e}\}) &= A(\underline{l}; \underline{\beta}(x); \{\underline{\gamma}\}) \\ &= \{\alpha(\underline{l}; \underline{\beta}; \underline{\gamma})\}(x) = \{\delta\}(x) \end{aligned}$$

as required.

Secondly suppose that C is obtained by an allowable recursion from A and B as in Sect. 4. Suppose also that A and B have indicial functions α and β , respectively, so that

$$\begin{aligned} A(\underline{l}; \underline{u}(x); \{\underline{d}\}) &= \{\alpha(\underline{l}; \underline{u}; \underline{d})\}(x) \\ B(\underline{l}; \underline{u}(x), \underline{v}(x); \{\underline{d}\}) &= \{\beta(\underline{l}; \underline{u}, \underline{v}; \underline{d})\}(x) \end{aligned}$$

for all appropriate $\underline{l}; \underline{u}, \underline{v}; \underline{d}$ and x . Using the same distinguished control position as in the construction of C , let γ be the primitive recursive function given by

$$\begin{aligned} \gamma(l_1, \dots, 0, \dots, l_a; \underline{u}; \underline{d}) &= \alpha(l_1, \dots, 0, \dots, l_a; \underline{u}; \underline{d}) \\ \gamma(l_1, \dots, l+1, \dots, l_a; \underline{u}; \underline{d}) &= \beta(l_1, \dots, l, \dots, l_a; \underline{u}, \underline{v}; \underline{d}) \end{aligned}$$

where

$$\underline{v} = \gamma(l_1, \dots, l, \dots, l_a; \underline{u}; \underline{d}).$$

An obvious argument now shows that γ is indicial for C . $\square$

Notice how in this proof the construction of the indicial function simply mimics the construction of the parent functional. Notice also how the construction restrictions ensure that the introduced variable x does not appear in an unwanted position.

6. The Result

We now have enough machinery to show just how extensive primitive recursion can be.

6.1. Theorem. *Let A and B be Ξ -functionals of rank (a, b, c) and $(1 + l, m, 1 + n)$ respectively. Let*

$$\begin{aligned} \underline{a} &\in \mathbb{N}^a, & \underline{b} &\in \mathbb{N}^b, & \underline{c} &\in \mathbb{N}^c \\ \underline{l} &\in \mathbb{N}^l, & \underline{m} &\in \mathbb{N}^m, & \underline{n} &\in \mathbb{N}^n \end{aligned}$$

be fixed sequences of parameters, and let $f: \mathbb{N}^2 \rightarrow \mathbb{N}$ be the function defined by

$$\begin{aligned} f(0, x) &= A(\underline{a}; \underline{b}(x); \{\underline{c}\}) \\ f(r+1, x) &= B(r, \underline{l}; \underline{m}(x); f(r, \cdot), \{\underline{n}\}) \end{aligned}$$

for $r, x \in \mathbb{N}$. Then there is a primitive recursive function $\phi \in \mathbb{F}$, effectively obtainable from the constructions of A and B , such that

$$f(r, x) = \{\phi(r)\}(x)$$

for all $r, x \in \mathbb{N}$. In particular f is primitive recursive in $\{\cdot\} \cdot$, the catalogue function.

Proof. Let α and β be primitive recursive indicial functions of A and B , respectively, and consider $\phi \in \mathbb{F}$ defined by

$$\begin{aligned} \phi(0) &= \alpha(\underline{a}; \underline{b}; \underline{c}) \\ \phi(r+1) &= \beta(r, \underline{l}; \underline{m}; \phi(r), \underline{n}) \end{aligned}$$

for $r \in \mathbb{N}$. Clearly ϕ is primitive recursive, and

$$\begin{aligned} f(0, x) &= \{\phi(0)\}(x) \\ f(r, \cdot) &= \{\phi(r)\} \Rightarrow f(r+1, x) = \{\phi(r+1)\}(x) \end{aligned}$$

for all $r, x \in \mathbb{N}$, hence the result. $\square$

Since the catalogue function is primitive recursive in $\mathcal{E}$, this shows that the defined f is primitive recursive in $\mathcal{E}$. However in most applications of this result, $\mathcal{E}$ is actually a function (i.e., $r=0$) which is trivially primitive recursive in the various given (initial) functions. Thus f is also primitive recursive in these given functions.

7. Two Examples

For the first example let us see how we reduce the mode of construction (2) (of the introduction) to primitive recursion.

We take a base functional $\mathcal{E}$ of type $(2, 2, 0)$ (so in fact $\mathcal{E}$ is a function) given by

$$\mathcal{E}(l, r; u, v) = \begin{cases} g(u) & \text{if } l=0 \\ h(r, u, v) & \text{if } l=1 \\ k(r, u) & \text{if } l \geq 2, \end{cases}$$

for $l, r, u, v \in \mathbb{N}$. The three functionals R, S, T where

$$\begin{aligned} R(r; u) &= \mathcal{E}(2, r; u, u) = k(r, u) \\ S(r; u; f) &= \text{App}(r; R(r; u); f) = f(k(r, u)) \\ T(r; u; f) &= \mathcal{E}(1, r; u, S(r; u; f)) = h(r, u, f(k(r, u))) \end{aligned}$$

are all allowable composites of initial $\mathcal{E}$ -functionals, so are themselves $\mathcal{E}$ -functions. The reader should go through the formal construction of these three, especially S , to make sure he understands the restrictions in allowable composition.

Note that (2) can now be rephrased as

$$\begin{aligned} f(0, x) &= \Xi(0, 0; 0(x), 0(x)) \\ f(r+1, x) &= T(r; 0(x); f(r, \cdot)) \end{aligned}$$

so that Theorem 6.1 is applicable.

The indicial functions of R , S , and T are given by

$$\begin{aligned} \varrho(r; u) &= \#(2, r, u, u) \\ \sigma(r; u; d) &= d * \varrho(r; u) \\ \tau(r; u; d) &= \#(1, r, u, \sigma(r; u; d)) \end{aligned}$$

and these are clearly primitive recursive. Note also that if we set

$$\begin{aligned} a &= \#(0, 0, 0, 0) \\ \kappa(r) &= \varrho(r; 0) = \#(2, r, 0, 0) \\ \eta(r, v) &= \#(1, r, 0, v) \end{aligned}$$

then

$$\begin{aligned} g(x) &= \{a\}(x) \\ h(r, x, v(x)) &= \{\eta(r; v)\}(x) \\ k(r, x) &= \{\kappa(r)\}(x) \end{aligned}$$

for all $r, x \in \mathbb{N}$. Thus if we define ϕ by

$$\begin{aligned} \phi(0) &= a \\ \phi(r+1) &= \eta(r; \phi(r) * \kappa(r)) = \tau(r; 0; \phi(r)) \end{aligned}$$

then $f(r, x) = \{\phi(r)\}(x)$.

I suggest that the reader compares how straightforward, even obvious, this construction is compared with any of the usual reductions of (2) to primitive recursion.

In the second example we illustrate the facility for constructing Ξ -functionals by allowable recursion.

Suppose the base functional Ξ has rank $(1, 1, 0)$ (so again it is a function).

Using the application functionals we easily check that the iteration functional given by

$$\text{It}(r; u; f) = f^{r+1}(u)$$

is a Ξ -functional (no matter what Ξ is). If we set

$$d^r = d * d * \dots * d \quad (r \text{ terms})$$

then we easily check that the indicial function ι of It is given by

$$\iota(r; u; d) = d^{r+1} * u.$$

Now let A be the composite given by

$$A(r; u; f) = \Xi(r+1; \text{App}(r; u; \text{It}(r; \cdot; f))).$$

This is a $\mathcal{E}$ -functional with indicial function α given by

$$\alpha(r; u; d) = \#(r+1, \iota(r; 0; d) * u) = \#(r+1, d^{r+1} * u)$$

(where this last equality holds since $*$ is associative). Finally let B be the $\mathcal{E}$ -functional given by

$$\begin{aligned} B(0; u; f) &= \mathcal{E}(0; f(u)) \\ B(r+1; u; f) &= A(r; B(r; u; f); f). \end{aligned}$$

The corresponding indicial function β is given by

$$\begin{aligned} \beta(0; u; d) &= \#(0, d * u) \\ \beta(r+1; u; d) &= \alpha(r; \beta(r; u; d); d). \end{aligned}$$

Theorem 6.1 now shows us that the function f given by

$$\begin{aligned} f(0, x) &= \mathcal{E}(0; x) \\ f(r+1, x) &= B(r; x; f(r, \cdot)) \end{aligned}$$

is primitive recursive in $\mathcal{E}$. It is interesting to see what this function is.

Writing

$$\xi_r \text{ for } \mathcal{E}(r; \cdot), \quad f_r \text{ for } f(r, \cdot)$$

we find that

$$A(r; \cdot; f) = \xi_{r+1} f^{r+1}$$

and

$$B(r; \cdot; f) = \xi_r f^r \xi_{r-1} f^{r-1} \dots \xi_1 f \xi_0 f$$

so that

$$\begin{aligned} f_0 &= \xi_0 \\ f_1 &= \xi_0 f_0 = \xi_0^2 \\ f_2 &= \xi_1 f_1 \xi_0 f_0 = \xi_1 \xi_0^5 \\ f_3 &= \xi_2 f_2^2 \xi_1 f_2 \xi_0 f_2 = \xi_2 \xi_1 \xi_0^5 \xi_1 \xi_0^5 \xi_1^2 \xi_0^6 \xi_1 \xi_0^5 \\ f_4 &= \xi_3 f_3^3 \xi_2 f_3^2 \xi_1 f_3 \xi_0 f_3 = \xi_3 \xi_2 \xi_1 \dots \xi_1 \xi_0^5 \end{aligned}$$

etc.

References

1. Hinman, P.G.: Recursion-theoretic hierarchies. Berlin Heidelberg New York: Springer 1978
2. Péter, R.: Recursive functions. New York London: Academic Press 1967
3. Schwichtenberg, H.: Eine Klassifikation der ε_0 -rekursiven Funktionen. Z. Math. Logik 17, 61–74 (1971)

Received February 2, 1988/in revised form May 9, 1988